CLASE 26

Virtual Environments + pip

Organizando dependencias como profesionales

- Por qué necesitamos entornos virtuales
- Crear y activar entornos con venv
- Instalar paquetes con pip
- requirements.txt - compartir dependencias
- Estructura de proyecto profesional

Después de hoy, sus proyectos serán portables y organizados

🚨 El problema: Conflicto de dependencias

Escenario real:

Proyecto A necesita: requests versión 2.25
Proyecto B necesita: requests versión 2.31

Si instalas ambas en tu computadora... ¡Solo puede haber UNA versión!

Otro problema clásico: "En mi computadora funciona"

Tú: "Funciona perfecto"
Tu compañero: "A mí me da error"
Motivo: No tiene los mismos paquetes instalados

Solución: Cada proyecto tiene su PROPIO entorno aislado

🏠 ¿Qué es un entorno virtual?

Analogía: Cada proyecto es una CASA

Sin entornos virtuales:

Todos comparten UN armario (Python global)
Si compras muebles nuevos, afectan a todas
Caos total

Con entornos virtuales:

Cada proyecto tiene su PROPIO armario
Proyecto A: sus paquetes (requests 2.25)
Proyecto B: sus paquetes (requests 2.31)
No se mezclan. Paz total

Técnicamente:

- Es una carpeta con una copia de Python
- Y su propia carpeta de paquetes instalados
- Aislado del Python del sistema

venv = virtual environment = entorno virtual (viene incluido con Python 3)

Flujo de trabajo con entornos virtuales

Proyecto NUEVO:

1. Crear carpeta del proyecto
2. Crear entorno virtual (una sola vez)
3. Activar el entorno
4. Instalar dependencias con pip
5. Trabajar en el proyecto
6. Al terminar: desactivar (opcional)

Proyecto EXISTENTE:

1. Navegar a la carpeta
2. Activar el entorno
3. Trabajar

El entorno ya existe, solo hay que activarlo

El paso 2 (crear el entorno) solo se hace UNA VEZ por proyecto

🔨 Paso 1: Crear el entorno virtual

```
python -m venv nombre_del_entorno
```

Convención: llamarlo "venv" o ".venv" → python -m venv venv

¿Qué hace este comando?

- Crea una carpeta llamada "venv"
- Dentro hay una copia de Python
- Y una carpeta vacía para paquetes

```
mi_proyecto/
  └── venv/
      ├── Scripts/     (Windows)
      ├── Lib/         (paquetes)
      └── pyvenv.cfg
```

Hagámoslo juntos en la terminal

⚡ Paso 2: Activar el entorno virtual

El entorno existe, pero hay que "entrar" a él

Windows (CMD):

```
venv\Scripts\activate
```

Windows (PowerShell):

```
venv\Scripts\Activate.ps1
```

Mac / Linux:

```
source venv/bin/activate
```

¿Cómo saber si está activado?

```
(venv) C:\Users\nombre\mi_proyecto>
```

Este (venv) al inicio indica que el entorno está ACTIVO

Siempre verificar que aparezca (venv) antes de trabajar

Desactivar el entorno

```
deactivate
```

Antes:

```
(venv) C:\Users\nombre\mi_proyecto>
```

Después:

```
C:\Users\nombre\mi_proyecto>
```

(sin el (venv) al inicio)

¿Cuándo desactivar?

- Cuando terminas de trabajar en el proyecto
- Cuando quieres cambiar a otro proyecto
- Es opcional al final del día (cerrar terminal también lo hace)

Desactivar NO borra el entorno. Solo "sales" de él. La próxima vez, activas de nuevo.

🔍 Verificar que todo está bien

Ver qué Python usamos:

```
where python    (Windows)
which python    (Mac/Linux)
```

Ver paquetes instalados:

```
pip list
```

En un entorno nuevo, solo debería haber:

- pip (el instalador de paquetes)
- setuptools (herramientas de instalación)

Si ven muchos paquetes, algo está mal (probablemente no está activado el entorno)

Entorno nuevo = pip list muestra casi nada

pip install - Instalar paquetes

pip = Package Installer for Python

Instalar un paquete:

```
pip install nombre_paquete
```

Instalar versión específica:

```
pip install requests==2.31.0
```

Instalar múltiples:

```
pip install requests pytest flask
```

¿De dónde vienen los paquetes?

PyPI = Python Package Index (pypi.org) - Miles de paquetes gratuitos

Con el entorno activado, pip instala SOLO en ese entorno

Otros comandos útiles de pip

Ver paquetes instalados:

```
pip list
```

Info de un paquete:

```
pip show requests
```

Actualizar un paquete:

```
pip install --upgrade requests
```

Desinstalar un paquete:

```
pip uninstall requests
```

Para buscar paquetes: usar https://pypi.org (pip search ya no funciona bien)

BREAK - 10 MINUTOS

Volvemos con:

requirements.txt
Estructura de proyecto profesional

❄️ pip freeze - Exportar dependencias

¿Cómo compartir qué paquetes necesita tu proyecto?

```
pip freeze

certifi==2023.7.22
charset-normalizer==3.2.0
pytest==7.4.0
requests==2.31.0
...
```

La magia: guardar en archivo

```
pip freeze > requirements.txt
```

Crea un archivo con TODAS las dependencias. Se comparte vía git.

El flujo: Tu proyecto → pip freeze → requirements.txt → Otro dev lo instala

Otros clonan tu repo, crean su venv, y ejecutan pip install -r requirements.txt para tener exactamente los mismos paquetes con las mismas versiones.

requirements.txt = la "receta" de tu proyecto

Instalar desde requirements.txt

Cuando alguien clona tu proyecto:

1. git clone <url_del_proyecto>
2. cd proyecto
3. python -m venv venv
4. venv\Scripts\activate
5. `pip install -r requirements.txt` ← La magia

-r = "read from file" = leer del archivo. Instala TODO lo que está en la lista.

Resultado: Todos tienen los mismos paquetes, con las mismas versiones. El proyecto funciona igual para todos.

requirements.txt va en git | La carpeta venv/ NO va en git

🚫 .gitignore - Qué NO subir a git

La carpeta venv/ NO debe subir a git:

- Es pesada (100+ MB)
- Es específica de cada SO
- Se puede recrear con requirements.txt

.gitignore:

```
# Entorno virtual
venv/
.venv/

# Python
__pycache__/
*.pyc

# IDE
.vscode/
.idea/
```

Lo que SÍ sube a git:

- requirements.txt (la receta)
- Tu código (.py)
- README.md, .gitignore, etc.

Lo que NO sube:

- venv/ (se recrea)
- __pycache__/ (auto-generado)
- .env (variables secretas)

📁 Estructura de proyecto profesional

```
mi_proyecto/
│
├── venv/                ← Entorno virtual (ignorado)
│
├── src/                 ← Tu código
│   ├── __init__.py
│   └── main.py
│
├── tests/               ← Tests
│   └── test_main.py
│
├── requirements.txt     ← Dependencias
├── .gitignore           ← Qué ignorar
└── README.md            ← Documentación
```

src/
Tu código fuente va aquí

tests/
Tests separados del código

requirements.txt
La receta del proyecto

.gitignore
Protege tu repo

README.md
Lo primero que ven otros

Esta estructura es estándar en la industria

✏️ PROYECTO: Setup de proyecto completo

Crear un proyecto desde cero con:

1. Estructura de carpetas (src/, tests/)
2. Entorno virtual activado
3. Dependencias: requests + pytest
4. requirements.txt generado
5. .gitignore configurado
6. README.md con instrucciones

El README debe incluir:

- Nombre y descripción del proyecto
- Cómo instalar (crear venv, instalar requirements)
- Cómo ejecutar y cómo correr tests

BONUS:

Crear un script simple que use requests y un test que lo pruebe

🎯 Revisión: Checklist de proyecto

- ✅ Estructura correcta: src/, tests/, venv/, archivos config
- ✅ Entorno virtual funciona: (venv) visible en terminal
- ✅ pip list muestra requests y pytest
- ✅ python src/main.py ejecuta sin errores
- ✅ pytest tests/ pasa los tests
- ✅ requirements.txt actualizado con pip freeze
- ✅ README claro con instrucciones de instalación, ejecución y tests
- ✅ .gitignore configurado (venv/ ignorado)

Este es el estándar de la industria

Resumen - Virtual Environments + pip

Acción	Comando
Crear entorno	`python -m venv venv`
Activar (Windows)	`venv\Scripts\activate`
Desactivar	`deactivate`
Instalar paquete	`pip install nombre`
Ver instalados	`pip list`
Exportar deps	`pip freeze > requirements.txt`
Instalar desde archivo	`pip install -r requirements.txt`

Flujo: Crear proyecto → Crear venv → Activar → Instalar → Trabajar

requirements.txt → Va a git (la receta)

venv/ → NO va a git (se recrea)

Próxima clase: APIs y Requests - "Consumir datos del mundo exterior"